Interdomain Routing & BGP

Autonomous Systems (AS)

- **Definition:** IP networks/routers under one admin with a common routing policy.
- **ASN:** Unique 16/32-bit AS identifier used in BGP.
- **AS relationships:**
  - **Customer / Provider:** Customer pays provider for transit.
  - **Full / Partial transit:** Full = global reachability; partial = limited reach.
  - **Peering:** Settlement-free; advertise own + customers only.
- **AS-level topology:** Common categories
  - **Tier-1:** No providers; global reach via peering.
  - **Tier-2:** Providers + peers (buys some transit).
  - **Stub AS:** Single provider; no downstream customers.

Border Gateway Protocol (BGP)

- **Definition:** Inter-AS routing protocol.
- **Path-vector:** Advertises AS_PATH; reject if own ASN appears.
- **Policy + attributes:** LOCAL_PREF, MED, AS_PATH, NEXT_HOP, etc.

BGP message flow (5 stages)

1. **TCP setup:** Session on TCP/179.
2. **OPEN:** Exchange ASN, hold-time, capabilities.
3. **Initial UPDATE:** Send NLRI + attributes.
4. **Decision:** Import policy + best-path selection.
5. **Steady state:** KEEPALIVE + incremental UPDATE/NOTIFICATION.

iBGP, eBGP and IGP interaction

- **eBGP:** Between ASes; increments AS_PATH, may change NEXT_HOP.
- **iBGP:** Within AS; no AS_PATH prepend; needs full mesh or route-reflectors.
- **IGP:** Provides internal reachability to BGP NEXT_HOP.

BGP Stability

- **Stable state:** Routes converge with no further changes.
- **Safety theorem (informal):** Under Gao-Rexford, BGP converges to a stable outcome.
- **Gao-Rexford conditions (commercial routing)**
  - **Topology:** Customer-provider edges form a DAG; peers are lateral.
  - **Preference:** Customer > peer > provider.
  - **Export:** Export all routes to customers; only customer routes to peers/providers.

Mathematical Background

Linear Algebra

- **Subspaces of** $A \in \mathbb{R}^{m\times n}$:
  - **Column space:** $\mathcal{C}(A) = \{Ax : x \in \mathbb{R}^n\}$
  - **Null space:** $\mathcal{N}(A) = \{x : Ax = 0\}$
  - **Row space:** $\mathcal{C}(A^T) \subseteq \mathbb{R}^n$
- **Rank-nullity theorem:** $\dim \mathcal{C}(A) + \dim \mathcal{N}(A) = n$
- **Fundamental decompositions:**
$$\mathbb{R}^m = \mathcal{C}(A) \oplus \mathcal{N}(A^T), \qquad \mathbb{R}^n = \mathcal{C}(A^T) \oplus \mathcal{N}(A)$$
- **Invertibility ($n \times n$ case):** The following are equivalent:
(1) $\det(A) \neq 0$, (2) $\operatorname{rank}(A) = n$,
(3) $\ker(A) = \{0\}$, (4) $\operatorname{im}(A) = \mathbb{R}^n$
- **Orthogonal projection matrix** P:
$$P^2 = P, \quad P^T = P, \quad (I - P)P = 0$$
- **Best-approximation property:** For a subspace V and any $x \in \mathbb{R}^m$,
$$\|x - Px\| \le \|x - v\| \quad \forall v \in V$$
- **Constructing P from an orthonormal basis** $\{v_i\}_{i=1}^k$:
$$P = \sum_{i=1}^k v_i v_i^T$$
- **Projection onto a single vector** u:
$$P_u(v) = \frac{\langle v, u\rangle}{\|u\|^2} u$$
- **Inverse of a 2×2 matrix:**
$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad A^{-1} = \frac{1}{\det A}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$
- **Orthogonal matrices:** $A^TA = AA^T = I$, hence $\det(A) = \pm 1$.

Matrix Decompositions

- **Eigenvalue Decomposition (EVD):** For symmetric $A = A^T \in \mathbb{R}^{n\times n}$: $A = U\Lambda U^T$, where U is orthogonal, $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$ (λ_i are eigenvalues). $A^k = U\Lambda^k U^T$, $A^{-1} = U\Lambda^{-1}U^T$ if all $\lambda_i \neq 0$.
- **Singular Value Decomposition (SVD):** (For any $A \in \mathbb{R}^{m\times n}$): $A = U\Sigma V^T$, where U and V are orthogonal, $\Sigma = \operatorname{diag}(\sigma_1, \dots, \sigma_r)$ (σ_i are singular values, $r = \operatorname{rank}(A)$). $A^TA = V\Sigma^2V^T$, so $\sigma_i^2 \ge 0$ are eigenvalues of A^TA.

Positive (Semi-) Definite Matrices

- **Positive semi-definite (PSD):** $A = A^T$ and $x^TAx \ge 0$ for all $x \in \mathbb{R}^n$
- **Equivalent characterizations:**
  - All eigenvalues $\lambda_i \ge 0$
  - There exists B such that $A = B^TB$

Probability

- **Covariance transformation:** If z is a random vector with $\mathbb{E}[z] = 0$, then $\operatorname{Var}(Az) = A\operatorname{Var}(z)A^T$

Calculus

- **Exponential bound:** $1 - x \le e^{-x}, \quad \forall x \in \mathbb{R}$
- **Gradient of a scalar** $f : \mathbb{R}^n \to \mathbb{R}$:
$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n}\right)^T$$
- **Jacobian of a vector** $f : \mathbb{R}^n \to \mathbb{R}^m$:
$$J_x(f) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_1} \\ \vdots & \cdots & \vdots \\ \frac{\partial f_1}{\partial x_d} & \cdots & \frac{\partial f_n}{\partial x_d} \end{pmatrix} = [\nabla f_1 \ \cdots \ \nabla f_m]$$
- **Chain rule:** $J_x(g \circ f) = J_{f(x)}(g) \cdot J_x(f)$
- **Hessian of a scalar** f:
$$H(f) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \cdots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$
- **Taylor expansions about** x_0:
$$f(x) \approx f(x_0) + \langle \nabla f(x_0), x - x_0 \rangle \quad \text{(1st order)},$$
$$f(x) \approx f(x_0) + \langle \nabla f(x_0), x - x_0 \rangle + \tfrac{1}{2}(x - x_0)^T H(f(x_0))(x - x_0) \quad \text{(2nd order)}.$$
- **Some common gradients / Jacobians:**
  - $\nabla\left(\frac{1}{2}\|Ax - y\|^2\right) = A^T(Ax - y)$
  - $\nabla(\|w\|^2) = 2w$
  - $J_w(Aw) = A$

Convexity

- **Convexity via Hessian:**
  - $H(f) \succeq 0$ (PSD) $\implies f$ is convex
  - $H(f) \preceq 0$ (NSD) $\implies f$ is concave
- **Convex set:** C is convex if $\forall u, v \in C,\ \forall \alpha \in [0, 1]:\ \alpha u + (1 - \alpha)v \in C$.
e.g. Unit Ball, Halfspace, Intersection or sum of convex sets.
- **Convex function:** f is convex on C if
$\forall u, v \in C,\ \forall \alpha \in [0, 1]:\ f(\alpha u + (1 - \alpha)v) \le \alpha f(u) + (1 - \alpha)f(v)$.
- **Preserved properties:**
  - **Non-negative sum:** $f = \sum a_i f_i$ is convex if f_i convex, $a_i \ge 0$.
  - **Affine composition:** $g(x) = f(Ax + b)$ is convex if f convex.
  - **Maximization:** $g(x) = \sup_{y \in A} f(x, y)$ is convex if $f(x, y)$ convex in x for each y.

Mathematical Background

Linear Algebra

- **Subspaces of** $A \in \mathbb{R}^{m\times n}$:
  - **Column space:** $\mathcal{C}(A) = \{Ax : x \in \mathbb{R}^n\}$
  - **Null space:** $\mathcal{N}(A) = \{x : Ax = 0\}$
  - **Row space:** $\mathcal{C}(A^T) \subseteq \mathbb{R}^n$
- **Rank-nullity theorem:** $\dim \mathcal{C}(A) + \dim \mathcal{N}(A) = n$
- **Fundamental decompositions:**
$$\mathbb{R}^m = \mathcal{C}(A) \oplus \mathcal{N}(A^T), \qquad \mathbb{R}^n = \mathcal{C}(A^T) \oplus \mathcal{N}(A)$$
- **Invertibility ($n \times n$ case):** The following are equivalent:
(1) $\det(A) \neq 0$, (2) $\operatorname{rank}(A) = n$,
(3) $\ker(A) = \{0\}$, (4) $\operatorname{im}(A) = \mathbb{R}^n$
- **Orthogonal projection matrix** P:
$$P^2 = P, \quad P^T = P, \quad (I - P)P = 0$$
- **Best-approximation property:** For a subspace V and any $x \in \mathbb{R}^m$,
$$\|x - Px\| \le \|x - v\| \quad \forall v \in V$$
- **Constructing P from an orthonormal basis** $\{v_i\}_{i=1}^k$:
$$P = \sum_{i=1}^k v_i v_i^T$$
- **Projection onto a single vector** u:
$$P_u(v) = \frac{\langle v, u\rangle}{\|u\|^2} u$$
- **Inverse of a 2×2 matrix:**
$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad A^{-1} = \frac{1}{\det A}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$
- **Orthogonal matrices:** $A^TA = AA^T = I$, hence $\det(A) = \pm 1$.

Matrix Decompositions

- **Eigenvalue Decomposition (EVD):** For symmetric $A = A^T \in \mathbb{R}^{n\times n}$: $A = U\Lambda U^T$, where U is orthogonal, $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$ (λ_i are eigenvalues). $A^k = U\Lambda^k U^T$, $A^{-1} = U\Lambda^{-1}U^T$ if all $\lambda_i \neq 0$.
- **Singular Value Decomposition (SVD):** (For any $A \in \mathbb{R}^{m\times n}$): $A = U\Sigma V^T$, where U and V are orthogonal, $\Sigma = \operatorname{diag}(\sigma_1, \dots, \sigma_r)$ (σ_i are singular values, $r = \operatorname{rank}(A)$). $A^TA = V\Sigma^2V^T$, so $\sigma_i^2 \ge 0$ are eigenvalues of A^TA.

Positive (Semi-) Definite Matrices

- **Positive semi-definite (PSD):** $A = A^T$ and $x^TAx \ge 0$ for all $x \in \mathbb{R}^n$
- **Equivalent characterizations:**
  - All eigenvalues $\lambda_i \ge 0$
  - There exists B such that $A = B^TB$

Probability

- **Covariance transformation:** If z is a random vector with $\mathbb{E}[z] = 0$, then $\operatorname{Var}(Az) = A\operatorname{Var}(z)A^T$

Calculus

- **Exponential bound:** $1 - x \le e^{-x}, \quad \forall x \in \mathbb{R}$
- **Gradient of a scalar** $f : \mathbb{R}^n \to \mathbb{R}$:
$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n}\right)^T$$
- **Jacobian of a vector** $f : \mathbb{R}^n \to \mathbb{R}^m$:
$$J_x(f) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_1} \\ \vdots & \cdots & \vdots \\ \frac{\partial f_1}{\partial x_d} & \cdots & \frac{\partial f_n}{\partial x_d} \end{pmatrix} = [\nabla f_1 \ \cdots \ \nabla f_m]$$
- **Chain rule:** $J_x(g \circ f) = J_{f(x)}(g) \cdot J_x(f)$
- **Hessian of a scalar** f:
$$H(f) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \cdots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$
- **Taylor expansions about** x_0:
$$f(x) \approx f(x_0) + \langle \nabla f(x_0), x - x_0 \rangle \quad \text{(1st order)},$$
$$f(x) \approx f(x_0) + \langle \nabla f(x_0), x - x_0 \rangle + \tfrac{1}{2}(x - x_0)^T H(f(x_0))(x - x_0) \quad \text{(2nd order)}.$$
- **Some common gradients / Jacobians:**
  - $\nabla\left(\frac{1}{2}\|Ax - y\|^2\right) = A^T(Ax - y)$
  - $\nabla(\|w\|^2) = 2w$
  - $J_w(Aw) = A$

Convexity

- **Convexity via Hessian:**
  - $H(f) \succeq 0$ (PSD) $\implies f$ is convex
  - $H(f) \preceq 0$ (NSD) $\implies f$ is concave
- **Convex set:** C is convex if $\forall u, v \in C,\ \forall \alpha \in [0, 1]:\ \alpha u + (1 - \alpha)v \in C$.
e.g. Unit Ball, Halfspace, Intersection or sum of convex sets.
- **Convex function:** f is convex on C if
$\forall u, v \in C,\ \forall \alpha \in [0, 1]:\ f(\alpha u + (1 - \alpha)v) \le \alpha f(u) + (1 - \alpha)f(v)$.
- **Preserved properties:**
  - **Non-negative sum:** $f = \sum a_i f_i$ is convex if f_i convex, $a_i \ge 0$.
  - **Affine composition:** $g(x) = f(Ax + b)$ is convex if f convex.
  - **Maximization:** $g(x) = \sup_{y \in A} f(x, y)$ is convex if $f(x, y)$ convex in x for each y.

Mathematical Background

Linear Algebra

- **Subspaces of** $A \in \mathbb{R}^{m\times n}$:
  - **Column space:** $\mathcal{C}(A) = \{Ax : x \in \mathbb{R}^n\}$
  - **Null space:** $\mathcal{N}(A) = \{x : Ax = 0\}$
  - **Row space:** $\mathcal{C}(A^T) \subseteq \mathbb{R}^n$
- **Rank-nullity theorem:** $\dim \mathcal{C}(A) + \dim \mathcal{N}(A) = n$
- **Fundamental decompositions:**
$$\mathbb{R}^m = \mathcal{C}(A) \oplus \mathcal{N}(A^T), \qquad \mathbb{R}^n = \mathcal{C}(A^T) \oplus \mathcal{N}(A)$$
- **Invertibility ($n \times n$ case):** The following are equivalent:
(1) $\det(A) \neq 0$, (2) $\operatorname{rank}(A) = n$,
(3) $\ker(A) = \{0\}$, (4) $\operatorname{im}(A) = \mathbb{R}^n$
- **Orthogonal projection matrix** P:
$$P^2 = P, \quad P^T = P, \quad (I - P)P = 0$$
- **Best-approximation property:** For a subspace V and any $x \in \mathbb{R}^m$,
$$\|x - Px\| \le \|x - v\| \quad \forall v \in V$$
- **Constructing P from an orthonormal basis** $\{v_i\}_{i=1}^k$:
$$P = \sum_{i=1}^k v_i v_i^T$$
- **Projection onto a single vector** u:
$$P_u(v) = \frac{\langle v, u\rangle}{\|u\|^2} u$$
- **Inverse of a 2×2 matrix:**
$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad A^{-1} = \frac{1}{\det A}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$
- **Orthogonal matrices:** $A^TA = AA^T = I$, hence $\det(A) = \pm 1$.

Matrix Decompositions

- **Eigenvalue Decomposition (EVD):** For symmetric $A = A^T \in \mathbb{R}^{n\times n}$: $A = U\Lambda U^T$, where U is orthogonal, $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$ (λ_i are eigenvalues). $A^k = U\Lambda^k U^T$, $A^{-1} = U\Lambda^{-1}U^T$ if all $\lambda_i \neq 0$.
- **Singular Value Decomposition (SVD):** (For any $A \in \mathbb{R}^{m\times n}$): $A = U\Sigma V^T$, where U and V are orthogonal, $\Sigma = \operatorname{diag}(\sigma_1, \dots, \sigma_r)$ (σ_i are singular values, $r = \operatorname{rank}(A)$). $A^TA = V\Sigma^2V^T$, so $\sigma_i^2 \ge 0$ are eigenvalues of A^TA.

Positive (Semi-) Definite Matrices

- **Positive semi-definite (PSD):** $A = A^T$ and $x^TAx \ge 0$ for all $x \in \mathbb{R}^n$
- **Equivalent characterizations:**
  - All eigenvalues $\lambda_i \ge 0$
  - There exists B such that $A = B^TB$

Probability

- **Covariance transformation:** If z is a random vector with $\mathbb{E}[z] = 0$, then $\operatorname{Var}(Az) = A\operatorname{Var}(z)A^T$

Calculus

- **Exponential bound:** $1 - x \le e^{-x}, \quad \forall x \in \mathbb{R}$
- **Gradient of a scalar** $f : \mathbb{R}^n \to \mathbb{R}$:
$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n}\right)^T$$
- **Jacobian of a vector** $f : \mathbb{R}^n \to \mathbb{R}^m$:
$$J_x(f) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_1} \\ \vdots & \cdots & \vdots \\ \frac{\partial f_1}{\partial x_d} & \cdots & \frac{\partial f_n}{\partial x_d} \end{pmatrix} = [\nabla f_1 \ \cdots \ \nabla f_m]$$
- **Chain rule:** $J_x(g \circ f) = J_{f(x)}(g) \cdot J_x(f)$
- **Hessian of a scalar** f:
$$H(f) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \cdots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$
- **Taylor expansions about** x_0:
$$f(x) \approx f(x_0) + \langle \nabla f(x_0), x - x_0 \rangle \quad \text{(1st order)},$$
$$f(x) \approx f(x_0) + \langle \nabla f(x_0), x - x_0 \rangle + \tfrac{1}{2}(x - x_0)^T H(f(x_0))(x - x_0) \quad \text{(2nd order)}.$$
- **Some common gradients / Jacobians:**
  - $\nabla\left(\frac{1}{2}\|Ax - y\|^2\right) = A^T(Ax - y)$
  - $\nabla(\|w\|^2) = 2w$
  - $J_w(Aw) = A$

Convexity

- **Convexity via Hessian:**
  - $H(f) \succeq 0$ (PSD) $\implies f$ is convex

- $H(f) \preceq 0$ (NSD) $\implies$ f is concave
- **Convex set:** C is convex if $\forall u, v \in C, \ \forall \alpha \in [0,1]: \ \alpha u + (1-\alpha)v \in C$. e.g. Unit Ball, Halfspace, Intersection or sum of convex sets.
- **Convex function:** f is convex on C if $\forall u, v \in C, \ \forall \alpha \in [0,1]: \ f(\alpha u + (1-\alpha)v) \le \alpha f(u) + (1-\alpha) f(v)$.
- **Preserved properties:**
  - **Non-negative sum:** $f = \sum a_i f_i$ is convex if f_i convex, $a_i \ge 0$.
  - **Affine composition:** $g(x) = f(Ax+b)$ is convex if f convex.
  - **Maximization:** $g(x) = \sup_{y \in A} f(x,y)$ is convex if $f(x,y)$ convex in x for each y.

General Concepts

Modes

Oracle (full information), (model) **Evaluation**, **Training** (calibration)

Hat Notation

$\hat{f}, \hat{\theta}, \hat{y}$ = estimated/learned values. Goal: $\hat{f} \approx f$

Noise Model

$y_i = f(x_i) + z_i$ where often assume $z_i \sim \mathcal{N}(\mu, \sigma^2)$

Four Stages in Learning Problems

1. **Choosing Hypothesis Class:** e.g $\mathcal{H}_{lin}$, $\mathcal{H}_{poly}$
2. **Choosing Loss Function** (per sample), e.g.:
   - 0-1: $L(y, \hat{y}) = \mathbb{1}_{y \neq \hat{y}}$
   - Hinge: $L(y, \hat{y}) = \max(0, 1 - y\hat{y})$
   - Absolute: $L(y, \hat{y}) = |y - \hat{y}|$
   - Squared: $L(y, \hat{y}) = (y - \hat{y})^2$
3. **Choosing Learning Principle:** e.g. ERM (Empirical Risk Minimization, almost always used)
4. **Designing Learning Algorithm:** e.g. SVD, Gradient Descent, Random Forest, SVM

Risk Measures

- **True / Expected Risk** (Generalization Error): $R(h) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(h(x), y)]$
- **Empirical Risk:** $\hat{R}(h) = \frac{1}{n}\sum_{i=1}^{n} L(y_i, h(x_i))$
- **RSS** (Residual Sum of Squares): $\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
- **MSE** (Mean Squared Error): $\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

  Note: Empirical risk with squared loss = MSE

Bias-Variance Tradeoff

Bias: error caused by overly simplistic assumptions (underfitting).
Variance: error from sensitivity to training data (overfitting).
Tradeoff: More complex models reduce bias but increase variance.
Goal: find balance to minimize total error.

No Free Lunch (NFL) Theorem

No universally best algorithm. Must choose $\mathcal{H}_\theta$ with prior knowledge.

Hyperparameters

Set before training using prior knowledge. e.g. learning rate, batch size, epochs, model architecture

ML in Practice

- **Data Preprocessing:** Cleaning, normalization, augmentation
- **Model Selection:** Choosing the right model architecture
- **Hyperparameter Tuning:** Grid search, random search, Bayesian optimization
- **Evaluation:** Cross-validation, train-test split, performance metrics
- **Deployment:** Model serving, monitoring, updating
- **Data Contamination:** TODO
- **Analytical vs. Numerical Methods:** REVISIT Analytical methods (e.g. Normal Equations) provide exact and closed-form solutions, while numerical methods (e.g. Gradient Descent) approximate solutions, but are often more flexible and applicable to larger datasets.

Regression

Linear Regression

- **Hypothesis Class:** $\mathcal{H}_{lin} = \left\{ x \mapsto w_0 + x^\top w \mid w \in \mathbb{R}^d \right\}$

  With intercept trick: $\tilde{x} = [1; x] \Rightarrow \mathcal{H}_{lin} = \left\{ \tilde{x} \mapsto \langle w, \tilde{x} \rangle \mid w \in \mathbb{R}^{d+1} \right\}$
- **Learning Principle:** ERM with respect to squared loss (RSS)
- **Learning Algorithm** Minimize RSS:

$$\hat{w} = \arg\min_w \|y - Xw\|_2^2 = \arg\min_w \sum_{i=1}^{m} (y_i - x_i^\top w)^2$$

Solutions:

- **Normal Equation:** $X^\top X \hat{w} = X^\top y$
- **Closed-form Solution:** $\hat{w} = (X^\top X)^{-1} X^\top y$ if $X^\top X$ is invertible

  (Special case: $\hat{w} = X^{-1} y$ if X is square and invertible)
- **SVD Solution:** When $X^\top X$ is not invertible, use SVD of X to compute minimum-norm solution. **SVD Decomposition:** $X = U \Sigma V^\top$. **Pseudoinverse:** $X^\dagger = V \Sigma^\dagger U^\top$, where

$$(\Sigma^\dagger)_{i,i} = \begin{cases} \frac{1}{\sigma_i} & \text{if } \sigma_i > \epsilon \\ 0 & \text{otherwise} \end{cases} \qquad (\epsilon > 0 \text{ threshold})$$

  Solution: $\hat{w} = X^\dagger y$.

Polynomial Regression (Fitting)

- **Hypothesis Class:** $\mathcal{H}_{poly} = \{ x \mapsto \sum_{i=0}^{d} w_i x^i \mid w_i \in \mathbb{R} \}$
- **Loss Function:** Squared loss
- **Learning Principle:** ERM
- **Learning Algorithm:** Minimize RSS using polynomial features

  Note: High-degree polynomials reduce bias but increase variance
- **Feature Construction:** For m samples $x_1, \ldots, x_m$, construct the **Vandermonde matrix** X:

$$X = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \cdots & x_1^d \\ 1 & x_2^1 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_m^1 & x_m^2 & \cdots & x_m^d \end{pmatrix}$$

- **Solution:** Solve $\hat{w} = \arg\min_w \|y - Xw\|^2$ (normal equation or closed-form solution).

Classification

Evaluation Metrics

	Positive	Negative
True	TP	TN
False	FP	FN

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$
- **Precision:** $\frac{TP}{TP+FP}$, **Recall:** $\frac{TP}{TP+FN}$
- **Specificity:** $\frac{TN}{TN+FP}$, **F1:** $2 \cdot \frac{P \cdot R}{P+R}$
- **FPR:** $\frac{FP}{FP+TN}$, **FNR:** $\frac{FN}{FN+TP}$

Half-spaces

- **Hypothesis:** $\mathcal{H}_{half} = \{ x \mapsto \text{sign}(\langle x, w \rangle + b) \}$
- **Loss:** $L_s(h_w) = \sum_{i=1}^{m} \mathbb{1}_{[y_i \langle x_i, w \rangle < 0]}$
- **ERM:** $\arg\min_w$ s.t. $\text{diag}(y) \cdot X^T w \ge 1$

Baseline (Perceptron) Algorithm

Initialize: $w = \vec{0}$
while $\exists i$ s.t. $y_i \langle x_i, w \rangle \le 0$ **do**
  $w \leftarrow w + y_i x_i$
end while

Support Vector Machine (SVM)

- **Hypothesis:** $\mathcal{H}_{svm} = \{ x \mapsto \text{sign}(\langle x, w \rangle + b) \}$
- **Margin:** $D((b, w), x_i) = \min_{v: \langle v, w \rangle + b = 0} \|x_i - v\|$
- **If** $\|w\| = 1$, $D((b, w), x_i) = |\langle x_i, w \rangle + b|$
- **Hard-SVM:** $\arg\min_{w,b} \|w\|^2$ s.t. $y_i(\langle x_i, w \rangle + b) \ge 1$
- **Soft-SVM:** $\arg\min_{w,b} \lambda \|w\|^2 + \frac{1}{m}\sum_{i=1}^{m} \xi_i$ s.t. $y_i(\langle x_i, w \rangle + b) \ge 1 - \xi_i, \xi_i \ge 0$
- **Equivalent:**

  $\arg\min_w \lambda \|w\|^2 + \frac{1}{m}\sum_{i=1}^{m} \max(0, 1 - y_i(\langle x_i, w \rangle + b))$ (hinge loss)

  Margin = weighted norm of w

Logistic Regression

- **Hypothesis:** $h_w(x) = \sigma(w^\top x)$, where $\sigma(z) = \frac{1}{1+e^{-z}}$ (sigmoid)
- **Loss Function:** Logistic (cross-entropy) loss:

  $\ell(w) = -\sum_{i=1}^{m} [y_i \log h_w(x_i) + (1 - y_i) \log(1 - h_w(x_i))]$
- **Learning Principle:** ERM
- **Learning Algorithm:** Minimize loss using gradient descent (no closed-form solution)
- **Gradient:** $\nabla \ell(w) = \sum_{i=1}^{m} (h_w(x_i) - y_i) x_i$

  Note: Outputs are probabilities, decision by threshold (e.g., $h_w(x) > 0.5$)

K Nearest Neighbors (KNN)

- **Distance:** Euclidean: $\rho(x, z) = \sum_{j=1}^{d} (x_j - y_j)^2$
- **Prediction:** $\arg\max_{y \in \mathcal{Y}} \sum_{i=1}^{k} \mathbb{1}_{[y_{\pi(i)} = y]}$ (majority label among k nearest neighbors)
- **Algorithm:** For each sample, compute distance to all others, select k closest, predict by majority.
- **Complexity:** $O(md)$ (m = samples, d = features)
- **Bias-Variance:** Larger $k \to$ higher bias, lower variance (model less complex)

Classification Trees

- **Hypothesis:** $\mathcal{H}_{CT}^{k} = \left\{ x \mapsto \sum_{j=1}^{N} c_j \cdot \mathbb{1}_{[x \in B_j]} \right\}$
- N = number of leaves, k = max depth, B_j = partition (leaf), c_j = label for B_j
- **Prediction:** Assign x to the label of its leaf.
- **Leaf label:** $P_y^S(B) = \frac{1}{n_S(B)} \sum_{x_i \in B} \mathbb{1}_{[y_i = y]}$ (majority in leaf)
- **Tree Growth:** Recursively split data by features to maximize purity, until max depth or min samples per leaf.
- **Complexity:** $O(md2^k)$ (m = samples, d = features, k = depth)
- **Pruning:** After growing, prune to reduce variance: $\min_{T \subseteq T_0} [L_S(T) + \lambda |T|]$ (tradeoff size/loss)
- **Bias-Variance:** Deeper tree $\to$ lower bias, higher variance.

PAC Framework

PAC (Probably Approx. Correct) Learning

Assumptions:

- Realizable: target function $f \in \mathcal{H}$.
- No noise: $S \in (\mathcal{X} \times \mathcal{Y})^m = \{(x_i, f(x_i))\}_{i=1}^m$
- Loss function is 0-1 (binary classification).
- Data is i.i.d. from a fixed distribution D.

PAC Learnability: $\mathcal{H}$ is PAC learnable if $\forall \epsilon > 0, \forall \delta > 0, \exists m(\epsilon, \delta)$ s.t. for every distribution $\mathcal{D}$ and every $f \in \mathcal{H}$: $\mathbb{P}_{\mathcal{D}^m}\left(L_{\mathcal{D},f}(\mathcal{A}(S_m)) < \epsilon\right) > 1 - \delta$

Notes:

- The collection of PAC-learnable hypothesis classes is closed under complementation, and under finite union and finite intersection.

Agnostic PAC (APAC) Learning

Assumptions relaxation: We only keep the assumption that data is i.i.d. from a fixed distribution. Now, labels may have noise, true labeling function may not be in the hypothesis class, and the loss function can be any (not just 0-1).

APAC Learnability: A hypothesis class $\mathcal{H}$ is APAC learnable w.r.t. loss ℓ if there exists a function $\tilde{m}_{\mathcal{H}} : (0,1)^2 \to \mathbb{N}$ and a learning algorithm $\mathcal{A} : (\mathcal{X} \times \mathcal{Y})^m \to \mathcal{H}$ such that: For every $(\epsilon, \delta) \in (0,1)$, for every distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$, and for any $m \ge \tilde{m}_{\mathcal{H}}(\epsilon, \delta)$: $\mathbb{P}_{\mathcal{D}}\left(L_{\mathcal{D}}(\mathcal{A}(S_m)) < \min_{f \in \mathcal{H}} L_{\mathcal{D}}(f) + \epsilon\right) > 1 - \delta$

PAC vs. Agnostic PAC:

	PAC	Agnostic PAC
$\mathcal{D}$	$\mathcal{D}$ over $\mathcal{X}$	$\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$ No realizability assumption.
Train.	$x_i \sim \mathcal{D}, y_i = f(x_i)$	$(x_i, y_i) \sim \mathcal{D}$
Trgt	$f \in \mathcal{H}$	Possibly not in $\mathcal{H}$; labels may be stochastic.
Loss	0-1 loss	$\ell : \mathcal{H} \times (\mathcal{X}, \mathcal{Y}) \to \mathbb{R}_+$
Risk	$\mathbb{P}_{x \sim \mathcal{D}}[h(x) \neq f(x)]$	$\mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(h, (x, y))]$
Acc.	$\text{risk}(f_S) \le \epsilon$	$\text{risk}(f_S) \le \min_{f \in \mathcal{H}} \text{risk}(f) + \epsilon$

VC Dimension

Ensemble Methods

Bootstrapping

Bagging

Random Forests

Boosting

Regularization

Regularization Term: $\mathcal{R}$ penalizes model complexity. For $\lambda \ge 0$:

$$h_s = \arg\min_{h \in \mathcal{H}} F_s(h) + \lambda \mathcal{R}(h)$$

- $\lambda = 0$: No penalty, may overfit.
- $\lambda \to \infty$: Strong penalty, simpler hypothesis.
- **Tradeoff:** Larger $\lambda \to$ higher bias, lower variance.
- **Where to add regularization:** Add the term where total loss is computed for model selection (usually empirical risk).
- **What to regularize:** Regularize what controls model capacity/flexibility (e.g. weights in neural networks, coefficients in linear models).

Linear Regression Regularization

- **L0 Regularization (Best Subset Selection):** Penalizes number of nonzero coefficients.

$$\min_{w_0 \in \mathbb{R}, w \in \mathbb{R}^d} \left\{ \|w_0 \cdot \vec{1} + Xw - y\|^2 + \lambda \|w\|_0 \right\}$$

- **L1 Regularization (Lasso):** Promotes sparsity via absolute value penalty.

$$\min_{w_0 \in \mathbb{R}, w \in \mathbb{R}^d} \left\{ \|w_0 \cdot \vec{1} + Xw - y\|^2 + \lambda \|w\|_1 \right\}$$

- **L2 Regularization (Ridge):** Penalizes large coefficients via squared norm.

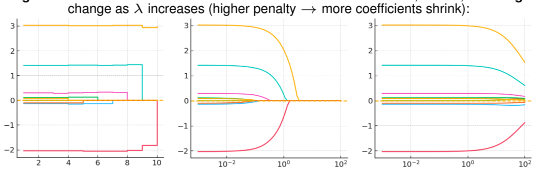
$$\min_{w_0 \in \mathbb{R}, w \in \mathbb{R}^d} \left\{ \|w_0 \cdot \vec{1} + Xw - y\|^2 + \lambda \|w\|_2^2 \right\}$$

- **Tikhonov Regularization:** Generalizes Ridge by penalizing $\|Lw\|^2$ for matrix $L \in \mathbb{R}^{k \times d}$.

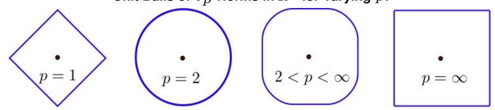
$$\hat{w}_L^{\text{Tik}} = \arg\min_{w \in \mathbb{R}^d} \left\{ \|Xw - y\|^2 + \|Lw\|_2^2 \right\}$$

  Special case: $L = \sqrt{\lambda} I$ reduces to Ridge regression.

Regularization Path: Shows how model coefficients for **Best Subset**, **Lasso**, and **Ridge** change as λ increases (higher penalty $\to$ more coefficients shrink):



Unit Balls of ℓ_p Norms in $\mathbb{R}^2$ for Varying p:



Hyperparameter Tuning

- **K-Fold Cross-Validation:**

Unsupervised Learning

Clustering

K-Means

- **Goal:** Partition m samples into k disjoint clusters so that points are *as close as possible* to their respective cluster centroid.
- **Objective:** Minimize $\sum_j \sum_{x \in C_j} \|x - \mu_j\|^2$ (sum of squared distances to centroids).
- **Optimal clustering is NP-hard:** Cannot be solved exactly in polynomial time; K-Means is a heuristic that finds a local minimum.

K-Means Algorithm (Lloyd's)

Input: $X = \{x_1, \ldots, x_m\} \subset \mathbb{R}^d, k \in \mathbb{N}_+$
Randomly choose initial centroids $\mu_1, \ldots, \mu_k$
repeat
  (Assignment)
  $C_j \leftarrow \{x_i \mid j = \arg\min_n \|x_i - \mu_n\|_2^2\} \quad \forall j$
  (Update) $\mu_j \leftarrow \frac{1}{|C_j|} \sum_{x \in C_j} x \quad \forall j$
until centroids unchanged **or** assignments no longer move points
Return: clusters $C_1, \ldots, C_k$

- **Convergence:** Guaranteed to terminate in $\mathcal{O}(m)$ iterations, each strictly reducing G, but only to a *local* minimum.
- **Sensitivity:** Result depends heavily on initial centroids;
- **Robustness:** Use *multiple random restarts* for better results.
- **Bias-variance trade-off:** Higher k reduces bias, increases variance.

Spectral Clustering

REVISIT

- **Goal:** Detect clusters based on the *connectivity structure* of the data rather than purely geometric proximity.
- **Affinity graph:** Build a weighted, undirected graph on samples $X = \{x_1, \ldots, x_m\} \subset \mathbb{R}^d$. For a bandwidth hyperparameter $\varepsilon > 0$,

$$A_{ij} = \exp\left(-\frac{\|x_i - x_j\|_2^2}{\varepsilon}\right) \quad \text{(often sparsified / } k\text{-NN).}$$

  $A \in \mathbb{R}^{m \times m}$ is the *affinity matrix*.
- **Graph Laplacian (unnormalised):**

$$D = \text{diag}\Big(\sum_j A_{ij}\Big), \quad L = D - A.$$

  (A normalised version $L_{\text{sym}} = D^{-1/2} L D^{-1/2}$ is common as well.)
- **Key spectral fact:** Number of connected components equals the multiplicity of eigenvalue 0 of L (spectral graph theory).

Spectral Clustering Algorithm (simplified)

Input: data X, bandwidth $\varepsilon > 0$, desired clusters k
Build affinity matrix $A_{ij} = \exp(-\|x_i - x_j\|_2^2 / \varepsilon)$
Compute degree matrix $D = \text{diag}(\sum_j A_{ij})$
Form unnormalised Laplacian $L = D - A$
Compute k eigenvectors $u_1, \ldots, u_k$ of L with *smallest* eigenvalues
Stack $U = [u_1, \ldots, u_k] \in \mathbb{R}^{m \times k}$ (i-th row $\to$ embedding of x_i)
Apply K-Means to the rows of U to obtain clusters $C_1, \ldots, C_k$
Return: $C_1, \ldots, C_k$

- **No random initialisation** of centroids (randomness only from graph sparsification or K-Means post-step).
- **Hyperparameters:** bandwidth ε and affinity type (heat kernel, k-NN graph, etc.).
- **Choosing** k: count eigenvalues of L (or L_{sym}) that are *close to zero* (eigengap heuristic).
- **Properties:**
  - Captures *non-convex* or *manifold* clusters.
  - More robust to noise than naïve connectivity methods.
  - ε acts as a bias–variance knob (large $\varepsilon \Rightarrow$ smoother, fewer clusters).

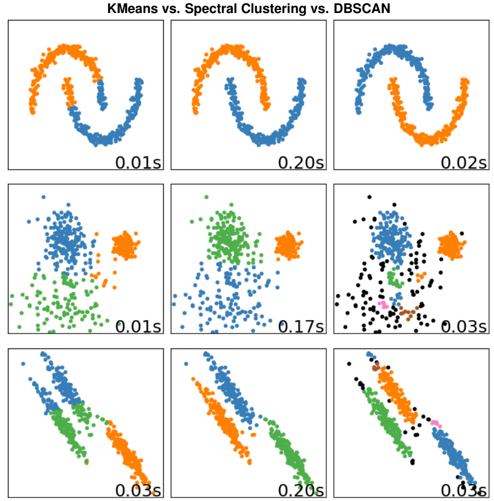
DBSCAN

Density-based clustering that groups together points that are close to each other.

Agglomerative Clustering

Hierarchical clustering that merges clusters based on distance.

KMeans vs. Spectral Clustering vs. DBSCAN



Dimensionality Reduction

PCA (Principal Component Analysis)

- **Goal:** Find k orthogonal directions (principal components) capturing maximal variance.
- **Formulation:** $W \in \mathbb{R}^{k \times d}$, $WW^T = I_k$
- **Optimization:** $w^* = \arg\max_{\{v_1, \ldots, v_k\}} \sum_{j=1}^{k} \text{Var}(v_j^T X)$
- **Covariance Matrix:** $\Sigma = \frac{1}{m}\sum_{i=1}^{m} (x_i - \bar{x})(x_i - \bar{x})^T$
- **Principal Components:** Top k eigenvectors of Σ (correspond to largest eigenvalues)
- **Projection:** $x \mapsto W^T(x - \bar{x})$
- **Properties:** $\|x - v\|_2 \ge \|x - Px\|_2$ for $v \notin \text{span}(u_1, \ldots, u_k)$
- **Explained Variance:** λ_j is variance explained by u_j; $\sum_j \lambda_j$ is total variance.

PCA Algorithm (Covariance-based)

Input: $X \in \mathbb{R}^{m \times d}$ *(design matrix with m samples, d features)*
Compute empirical mean: $\bar{x} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i$
Center the data: $Z \leftarrow X - \mathbf{1}\bar{x}^\top$
Compute covariance matrix: $\Sigma \leftarrow \frac{1}{m} Z^\top Z$
Eigen-decompose Σ; keep top k eigenvectors $u_1, \ldots, u_k$
Return: $u_1, \ldots, u_k$ (principal component directions)

Vectors $u_1, \ldots, u_k$ are orthonormal and maximise retained variance.

Gradient Based Learning

Basic Idea use an iterative optimization algorithm to minimize a loss function by updating parameters in the opposite direction of the gradient.

Sub-gradient Concepts

- **Sub-gradient:** v is a sub-gradient of f at w if $f(u) \geq f(w) + \langle v, u - w \rangle$ for all u.
- **Sub-differential:** $\partial f(w)$ is the set of all sub-gradients of f at w.
- **Convexity:** f is convex if $f(tv + (1-t)u) \leq tf(v) + (1-t)f(u)$ for $t \in [0, 1]$.
- **Examples:** Halfspace, norm, sum of convex functions.
- **Directional derivative:** $f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle$ for differentiable f.

Backtracking Line Search Algorithm

```
Initialize: η = 1
while f(x) + αη⟨∇f(x), Δx⟩ < f(x + ηΔx) do
    η ← βη    (β ∈ (0, 1))
end while
Return η
```

Sub-gradient Descent Algorithm

```
Initialize: w^(1) = 0
for t = 1, . . . , T do
    Choose v_t ∈ ∂f(w^(t))
    w^(t+1) ← w^(t) − η_t v_t
end for
Return w̄ = (1/T) Σ_{t=1}^T w^(t)
```

Stochastic Gradient Descent (SGD) Algorithm

```
Initialize: w^(1) = 0
for t = 1, . . . , T do
    Choose (x, y) ∼ D
    Choose v_t ∈ ∂ℓ(w^(t), (x, y))
    w^(t+1) ← w^(t) − η_t v_t
end for
Return w̄ = (1/T) Σ_{t=1}^T w^(t)
```

Neural Networks

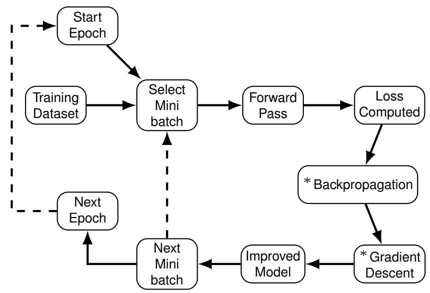
Backpropagation

"Backpropagation is the algorithm for determining how a single training example would like to nudge the weights and biases (Gradient-based update), not just in terms of whether they should increase or decrease, but also in terms of the relative proportions that would cause the most rapid decrease in the cost function (Gradient direction). A true gradient descent step would involve computing these adjustments for all tens of thousands of training examples and averaging the resulting gradients. However, this is computationally expensive, so instead, the training data is randomly subdivided into mini-batches, and each gradient descent step is computed with respect to one mini-batch at a time. By repeatedly going through all of the mini-batches (Epoch) and updating the weights accordingly, the network gradually converges toward a local minimum of the cost function. In doing so, the network ends up performing very well on the training examples."

Useful Terms:

- **Mini-batch:** A small subset of the training data used to compute the gradient for a single update step.
- **Batch Size:** The number of training examples in a mini-batch (hyperparameter).
- **Epoch:** One complete pass through the entire training dataset (via all mini-batches).
- **Dropout (rate):** A regularization technique where a fraction of neurons are randomly set to zero during a batch training to prevent overfitting. The rate is an hyperparameter (usually 50%).

Training Pipeline:



* *Backpropagation computes gradients; a gradient-based optimiser applies them over many mini-batches and epochs.*

FNNs (Feedforward Neural Networks)

CNNs (Convolutional Neural Networks)

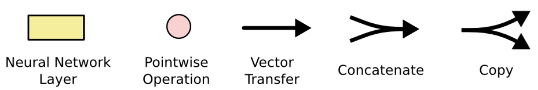
RNNs (Recurrent Neural Networks)

RNNs process sequential data, using previous steps to inform current predictions. Useful for text, audio, etc. and supports one/many-to-one/many sequence I/O patterns (e.g image → description is one-to-many).

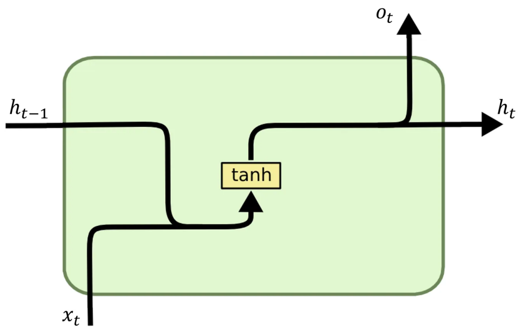
$h_t = \tanh(Uh_{t-1} + Vx_t)$

Basic RNN

Diagrams notation:

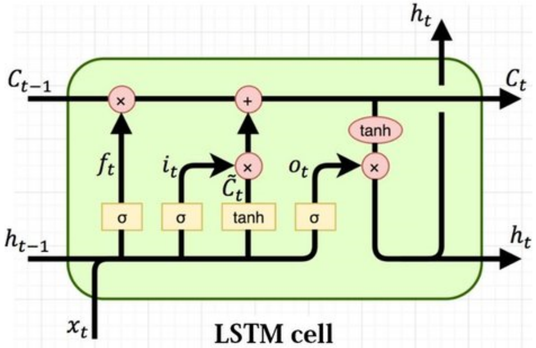


RNN Cell:



Long Short-Term Memory (LSTM)

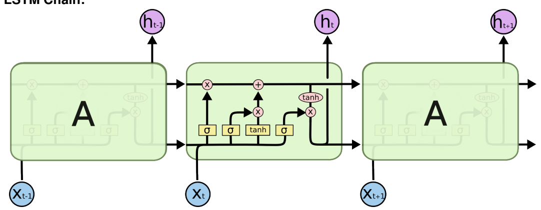
LSTMs are a type of RNN designed to remember information for long periods, addressing the vanishing gradient problem. They achieve this through a cell state (C_t) and three gates: input (i_t), forget (f_t), and output (o_t).



Where:

- $i_t = \sigma(x_t U^i + h_{t-1} W^i)$ Input gate, controls how much of the input to let through.
- $f_t = \sigma(x_t U^f + h_{t-1} W^f)$ Forget gate, controls how much of the previous cell state to keep.
- $o_t = \sigma(x_t U^o + h_{t-1} W^o)$ Output gate, controls how much of the cell state to output.
- $\tilde{c}_t = \tanh(x_t U^g + h_{t-1} W^g)$ Candidate cell state, potential new information to add to the cell state.
- $c_t = \sigma(f_t * C_{t-1} + i_t * \tilde{C}_t)$ Cell state, memory of the LSTM.
- $h_t = \tanh(C_t) * o_t$ Hidden state, output of the LSTM at time t.

LSTM Chain:



RNNs vs. CNNs

Property	RNN	CNN
Long-range mechanism	Gating/Attention	Pooling
Sequential	$+$	$-$
Parallelizable	$-$	$+$
Compute time for n tokens	$O(n \cdot L)$	$O(L)$
Layers required for the n-th token	1	$\log n$